

♥ Like

📌 Save

🔗 Share



@thedevspaceio

Regular Expression Cheatsheet



Creating regex

Regular expressions are patterns used to match character combinations in strings.

Literal syntax

js



```
1 const re = /hello/;  
2 re.test("hello world"); // true
```

RegExp constructor

js



```
1 const re = new RegExp("hello", "i"); // same as /hello/i  
2 re.test("HELLO world"); // true
```

Flags

Flag	Description
<code>g</code>	Global search.
<code>i</code>	Case-insensitive.
<code>m</code>	Multiline mode.
<code>s</code>	Dotall mode (<code>.</code> matches newlines).
<code>u</code>	Unicode mode.
<code>y</code>	Sticky mode.

`g`: Global

Finds all matches instead of stopping at the first.

```
js
```



```
1 "hello world".match(/o/g); // ["o", "o"]
2 "a1b2c3".replace(/\d/g, "x"); // "axbxcx"
```

i: Case-insensitive

Matches letters regardless of case.

js



```
1 /hello/i.test("HELLO"); // true
2 "Hello World".match(/hello/i); // ["Hello"]
```

m: Multiline

Changes `^` and `$` to match the start and end of each line instead of the whole string.

js



```
1 const text = "hello\nworld";
2 /^world$/m.test(text); // true
```

s: Dotall

Makes `.` match newline characters too.

js



```
1 const html = "<div>\n  hello\n</div>";
2 /<div>.*</div>/s.test(html); // true
```

u: Unicode

Enables full Unicode matching and lets you use `\u{}` escapes and Unicode property escapes.

js



```
1 /\u{1F600}/u.test("😄 "); // true
2 /\p{Emoji}/u.test("😄 "); // true
```

y: Sticky

Matches only from the `lastIndex` position and does not search further.

js



```
1 const re = /\d+/y;
2 re.lastIndex = 2;
3 "ab12cd".match(re); // ["12"]
4
5 re.lastIndex = 0;
6 "ab12cd".match(re); // null
```

Character classes

Pattern	Matches
<code>.</code>	Any character except newline (Any character with <code>s</code> flag).
<code>\d</code>	Any digit (<code>0-9</code>).
<code>\D</code>	Any non-digit.
<code>\w</code>	Any word character (<code>a-z</code> , <code>A-Z</code> , <code>0-9</code> , <code>_</code>).
<code>\W</code>	Any non-word character.
<code>\s</code>	Any whitespace character.
<code>\S</code>	Any non-whitespace character.
<code>[abc]</code>	Any character in the set.
<code>[^abc]</code>	Any character not in the set.
<code>[a-z]</code>	Any character in the range.

`.`: Any character

js

```
1 "a".match(/./); // ["a"]
2 "abc".match(/./g); // ["a", "b", "c"]
```

`\d` : Digit

js



```
1 "abc123".match(/\d+/); // ["123"]
2 "Room 42".match(/\d+/); // ["42"]
```

`\D` : Non-digit

js



```
1 "abc123".match(/\D+/); // ["abc"]
2 "2024!".match(/\D/); // ["!"]
```

`\w` : Word character

js



```
1 "hello_world".match(/\w+/); // ["hello_world"]
2 "user_123".match(/\w+/); // ["user_123"]
```

\W : Non-word character

js



```
1 "hello world".match(/\W/); // [" "]
2 "a@b".match(/\W/); //["@"]
```

\s : Whitespace

js



```
1 "a b".match(/\s/); // [" "]
2 "a\tb".match(/\s/); // ["\t"]
```

\S : Non-whitespace

js



```
1 " a".match(/\S/); // ["a"]
2 " hi".match(/\S+/); // ["hi"]
```

[abc] : Character set

js



```
1 "cat".match(/[aeiou]/); // ["a"]
2 "sky".match(/[aeiou]/); // null
```

[^abc] : Negated character set

js



```
1 "cat".match(/^[aeiou]/); // ["c"]
2 "aei".match(/^[aeiou]/); // null
```

[a-z] : Character range

js



```
1 "Hello".match(/[a-z]/); // ["e"]
2 "HELLO".match(/[a-z]/); // null
```

Quantifiers

Pattern	Meaning
<code>*</code>	Zero or more.
<code>+</code>	One or more.
<code>?</code>	Zero or one.
<code>{n}</code>	Exactly <code>n</code> times.
<code>{n,}</code>	At least <code>n</code> times.
<code>{n,m}</code>	Between <code>n</code> and <code>m</code> times.

`*`: Zero or more

js



```
1 "aaab".match(/a*/); // ["aaa"]
2 "b".match(/a*/); // [""]
```

`+`: One or more

js



```
1 "aaab".match(/a+/); // ["aaa"]
2 "b".match(/a+/); // null
```

? : Zero or one

js



```
1 "color".match(/colou?r/); // ["color"]
2 "colour".match(/colou?r/); // ["colour"]
```

{n} : Exactly n times

js



```
1 "123".match(/\d{3}/); // ["123"]
2 "12".match(/\d{3}/); // null
```

{n,} : At least n times

js



```
1 "12345".match(/\d{3,}/); // ["12345"]
2 "12".match(/\d{3,}/); // null
```

{n,m} : Between n and m times

js



```
1 "1234".match(/\d{2,4}/); // ["1234"]
2 "1".match(/\d{2,4}/); // null
```

Anchors

Pattern	Matches
<code>^</code>	Start of string (or line in <code>m</code> mode).
<code>\$</code>	End of string (or line in <code>m</code> mode).
<code>\b</code>	Word boundary.
<code>\B</code>	Non-word boundary.

`^`: Start of string

js



```
1 /^hello/.test("hello world"); // true
2 /^world/.test("hello world"); // false
```

`$`: End of string

js



```
1 /world$/.test("hello world"); // true
2 /hello$/.test("hello world"); // false
```

\b : Word boundary

js



```
1 /\bworld\b/.test("hello world"); // true
2 /\bworld\b/.test("helloworld"); // false
```

\B : Non-word boundary

js



```
1 /\Bworld\B/.test("helloworld"); // true
2 /\Bworld\B/.test("hello world"); // false
```

Groups

Pattern	Description
<code>(...)</code>	Capturing group. Stores the matched text for later use.
<code>(?:...)</code>	Non-capturing group. Groups without storing.
<code>(?<name>...)</code>	Named capturing group. Stores the match under a name.

Capturing group

js

```
1 const match = "2024-07-12".match(/(\d{4})-(\d{2})-(\d{2})/);
2 match[0]; // "2024-07-12"
3 match[1]; // "2024"
4 match[2]; // "07"
5 match[3]; // "12"
```

Non-capturing group

js



```
1 const match = "foo".match(/(?:foo|bar)/); // ["foo"]
2 match[0]; // "foo"
```

Named capturing group

js



```
1 const match = "2024-07-12".match(
2   /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/,
3 );
4 match.groups.year; // "2024"
```

Lookarounds

Pattern	Description
<code>(?=...)</code>	Positive lookahead.
<code>(?!...)</code>	Negative lookahead.
<code>(?<=...)</code>	Positive lookbehind.
<code>(?<!=...)</code>	Negative lookbehind.

`(?=...)` : Positive lookahead

Matches a group only if followed by another pattern.

js



```
1 /\d+(?=px)/.exec("100px"); // ["100"]
2 /\d+(?=px)/.exec("100em"); // null
```

`(?!...)` : Negative lookahead

Matches a group only if not followed by another pattern.

js



```
1 /\d+(?!px)/.exec("100em"); // ["100"]
2 /\d+(?!px)/.exec("100px"); // null
```

(?<=...) : Positive lookbehind

Matches a group only if preceded by another pattern.

js



```
1 /(?!<=\$)\d+/.exec("Price: $42"); // ["42"]  
2 /(?!<=\$)\d+/.exec("Price: 42"); // null
```

(?!...) : Negative lookbehind

Matches a group only if not preceded by another pattern.

js



```
1 /(?!<!\$)\d+/.exec("Price: 42"); // ["42"]  
2 /(?!<!\$)\d+/.exec("Price: $42"); // null
```

JavaScript regex methods

`.test()`

Returns `true` if the pattern matches.

js



```
1 /hello/.test("hello world"); // true
```

`.match()`

Returns an array of matches.

js



```
1 "abc123".match(/\d+/); // ["123"]
2 "a1b2c3".match(/\d/g); // ["1", "2", "3"]
```

`.matchAll()`

Returns an iterator of all matches with capture groups.

js



```
1 const matches = "a1b2".matchAll(/[a-z](\d)/g);
2 for (const match of matches) {
3   console.log(match[0], match[1]);
4 }
```

`.replace()`

Replaces the first match, or all matches with the `g` flag.

js



```
1 "hello world".replace(/world/, "JS"); // "hello JS"
2 "a b c".replace(/\s/g, "-"); // "a-b-c"
```

`.replaceAll()`

Replaces all matches without needing the `g` flag.

js



```
1 "a b c".replaceAll(" ", "-"); // "a-b-c"
```

`.search()`

Returns the index of the first match, or `-1`.

js



```
1 "hello world".search(/world/); // 6
```

`.split()`

Splits a string using a regex.

js



```
1 "a,b;c".split(/[;,|/]); // ["a", "b", "c"]
```